C-6 리눅스 어셈블리 프로그래밍

정병수
bsjung@samsung.com

1. 리눅스 어셈블리 프로그래밍의 기초

이번 강좌를 통해 리눅스 어셈블리 프로그래밍에 대해서 알아보고자 한다. 리눅스에서 어셈블리 프로그래밍을 한다는 것은 황량한 사막을 건너는 것과 같은 느낌이 들 것이다. 어쩌다가 좋은 문서를 발견하면 사막에서 오아시스를 만나는 것처럼 갈증을 어느 정도 해소해 주지만 조금 더 쉽고 자상한 초보자들을 위한 문서들이 부족함을 느낄 것이다.

이번 강좌는 이러한 초보자들을 위해 기초부터 공부할 수 있는 예제 위주의 직접 실행해볼 수 있는 좋은 길잡이가 될 것이다. 또 중급 사용자들을 위해 리눅스 어셈블리를 사용하여 조그마한 운영체제를 만들어 테스트할 수 있도록 할 것이다.

자, 이제 나와 같이 리눅스에서 어셈블리 프로그래밍을 하는 사막을 향해 낙타를 타고 여행을 떠나보자.



[그림 1-1] 리눅스 어셈블리 로고

1.1 리눅스 어셈블리 프로그래밍에 대하여

리눅스 어셈블리 프로그래밍을 하기 위해서는 다음과 같은 여러 가지 항목에 대해서 알아야 할 것이다.

우선 리눅스 어셈블리 프로그래밍을 하기 위해 필요한 툴을 알아보자.

1. as : 일명 GAS라고 하며, GNU 어셈블러이다.
2. ld : GNU 링커이다.
3. gdb : GNU 디버거이다.

이들 툴을 사용함에 있어, 우선 리눅스 시스템에 대한 전반적인 이해가 필요할 것이다. 즉, 만약에 자신의 리눅스 시스템에 GCC가 설치가 안되었다면 문제가 될 것이며, 필요한 라이브러리와 헤더 파일들이 시스템의 어느 곳에 위치하는지도 알아야 할 것이다.

우선 앞에 말한 3가지 툴들중 첫 번째와 두 번째를 이진파일 유틸리티(Binary Utility)라고 부르며, 어떤 시스템에 컴파일러를 만들 때 가장 먼저 포팅되는 것들이다. 만약 여러분이 전혀 새로운 CPU에서 어셈블리 프로그래밍을 하고 싶다면 이러한 이진파일 유틸리티부터 포팅해야 한다.

운영체제 만드는 일도 이와 비슷해서, 일단 이진파일 유틸리티와 C 컴파일러가 제공되어야 비로소 운영체제 만드는 일이 시작된다. 리눅스라는 운영체제도 어떻게 보면 Richard Stallman이 만든 GNU 어셈블러와 C 컴파일러로 만들어진 하나의 프로그램에 지나지 않을지 모른다.

일단 어셈블리어 프로그래밍은 운영체제에 많이 좌우되는데, 우리가 프로그래밍하는 환경은 현재 거의 모든 리눅스 프로그램들이 쓰는 ELF라는 실행파일 형식을 따른다고 가정한다.

리눅스에는 실행파일 형식으로 예전부터 3가지 이진형식을 지원했는데, 첫 번째는 a.out이라는 형식으로 기존의 BSD 유닉스에서 쓰던 형식이고, 두 번째는 이것을 개량한 coff 형식이고, 세 번째는 ELF 형식으로 MS 윈도우의 DLL과 같이 동적으로 이진 파일을 다룰 수 있게 만든 것이다.

우선 리눅스에서 어떤 프로그램이 실행되는 과정을 알아보는 것이 중요하다. 즉, 사용자가 쉘에서 프로그램을 실행하면, sys_execve()라는 함수가 실행파일을 실행하게 되는데, 커널은 이 함수에서 받은 실행파일을 do_execve()라는 함수를 사용하여 열어 파일 형식을 search_binary_handler()라는 함수로 찾는다. 만약 파일 형식이 ELF이면 load_elf_binary()를 사용하여, 시스템의 메모리에 올리고, 마침내 start_thread()를 사용하여, 커널에서 사용자가 넘긴 실행파일이 실행할 수 있도록 하고 있다.

어셈블리어로 프로그래밍하는 사람들은 load_elf_binary()에 의해 프로그램이 시스템의 메모리에 올라갈 때, 메모리의 어떤 번지에 올라가는지 궁금할텐데, i386의 경우 0x08048000번지가 이러한 ELF 파일의 첫 번째 번지가 되며, 계속해서, text 섹션과 data 섹션 그리고, stack 섹션이 존재하게 된다.

하지만 이러한 번지도 ELF 형식으로 결과물을 내는 GCC와 GAS가 있다면 굳이 알 필요가 없으며, text 섹션과 data 섹션, stack 섹션에다가 필요한 어셈블리어를 쓰고, GAS와 LD를 사용하여, ELF 형식의 실행파일을 만들어 주게 되면, 시스템 커널이 알아서, ELF로 된 실행파일을 시스템의 메모리 0x08048000번지에 로딩하게 되는 것이다.

1.2 GAS의 코딩 스타일

다음으로 알아봐야 할 것이 실제 GAS를 프로그래밍하기 전에 GAS에서 쓰는 코딩 스타일이다. GAS는 UNIX에서 쓰는 어셈블러이기 때문에, 모토롤라 m68k등에서 쓰는 이른바, "AT&T" 스타일로 된 어셈블리어를 어셈블리한다. 이에 반해 인텔 CPU와 Microsoft의 어셈블리어에서 쓰는 스타일을 "Intel" 스타일이라고 한다.

다음은 "AT&T"로 된 어셈블리어의 코딩 스타일의 특징을 나열한 것이다.

레지스터 이름은 %로 시작한다.
즉, 기존의 "Intel" 스타일로 된 eax나 dl 은 $eax나 $dl로 써야한다.

오퍼렌드의 순서는 소스 레지스터가 먼저 오고, 목적 레지스터가 나중에 온다.
만약 ax의 레지스터의 내용을 dx에 넣고자 하면, 기존의 "Intel" 스타일에서는 mov ax, dx 와 같이 해줘야 한다면, "AT& T" 스타일에서는 mov %dx, %ax가 된다.

오퍼렌드의 길이는 인스트럭션 이름의 첨가자로 부여된다.

첨가자 b는 8비트의 바이트고, w는 16비트의 워드, l은 32비트의 긴 정수를 의미한다.
이를테면 movb는 바이트 연산이며, movw는 16비트 워드 연산이 된다.

직접(immediate) 레지스터는 $로 시작한다.
이를테면 5라는 값을 eax 레지스터에 넣고 싶다면, 다음과 같이 한다.

```
movl $5, %eax
```

오퍼렌드에 전치사가 없으면, 이것은 메모리 주소를 의미한다.
예를 들어, movl $foo, %eax는 foo라는 변수의 주소를 %eax 레지스터에 넣는 인스트럭션이만, movl foo, %eax는 foo라는 변수의 내용을 %eax 레지스터에 넣는 인스트럭션이다.

인덱싱과 인다이렉션(indirection)은 괄호로 표시된다.
예를 들어, testb $0x80, 17(%ebp)는 %ebp에 지정된 포인터값으로부터 17번째 옵셋(offset)에 있는 바이트 값의 최상위비트를 테스트하는 명령어이다.

현재 GAS를 사용하는 MS의 MASM을 먼저 배운 어셈블리어 프로그램들이 위의 AT&T 방식으로 된 프로그램 스타일에 익숙하지 않는데, 현재 리눅스에서 NASM이라는 Intel 방식의 스타일을 지원하는 어셈블러도 있다. 하지만 NASM은 i386만 지원하여, GAS처럼 다양한 CPU를 지원하지 못하고 있다.

필자는 되도록 GAS를 쓰도록 장려하지만, 굳이 Intel 방식으로 프로그램하려면, NASM도 고려해볼만한 일이다. 하지만, 이번 강좌에서는 NASM보다는 GAS에 비중을 많이 두어서, i386과 다른 ARM, MIPS, PPC에도 적용할수 있도록 하려고 한다.

1.3 간단한 GAS 사용법

가장 간단한 GNU 어셈블리어 사용법은 역시 GCC에 어셈블리어 소스를 추출해내는 방법이다.

다음과 같이 간단한 GAS를 이용하여 i386머신에서 어셈블리어 프로그램을 해보자.

Assembly 코드 작성
가장 간단히 Assembly 언어를 만드는 방법은 다음과 같이 C언어에서 Assembly 언어를 추출해 내는 방법이다.

다음은 hello.c 이다.

```
#include <stdio.h>

void main()
{
        printf("Hello World\n");
}
```

다음은 C 소스에서 Assembler 소스를 얻는 방법이다.

```
% gcc -S hello.c
```

다음은 이렇게 해서 얻는 Assembler 소스이다.

```
        .file   1 "hello.c"
        .version        "01.01"
        .set noat
gcc2_compiled.:
__gnu_compiled_c:
.section        .rodata
        .align 3
$C32:
        .ascii "Hello World\12\0"
.text
        .align 3
        .globl main
        .ent main
main:
        ldgp $29,0($27)
main..ng:
        lda $30,-16($30)
        .frame $15,16,$26,0
        stq $26,0($30)
        stq $15,8($30)
        .mask 0x4008000,-16
        bis $30,$30,$15
        .prologue 1
        lda $16,$C32
        jsr $26,printf
        ldgp $29,0($26)
$33:
        bis $15,$15,$30
        ldq $26,0($30)
        ldq $15,8($30)
        addq $30,16,$30
        ret $31,($26),1
        .end main
        .ident  "GCC: (GNU) 2.7.2.1"
```

[리스트 1-1] GCC를 사용해서 얻은 hello.s

기계가 만든 코드라서 사용자들이 잘 알아보지 못하더라도 그냥 지나쳐라. (필자도 무슨 지렁이 글씨처럼 보인다. ^.^)

GAS로 컴파일 하기

그러면 hello.s라는 assembly 파일이 만들어 지는데,
이것을 GNU Assembler인 as로 Object 파일을 만든다.

```
% as -o hello.o hello.s
```

실행 파일 만들기

이제 로더인 LD를 사용하여 라이브러리를 합쳐 실행 파일을 만들어 주면 된다.
이때 gcc로 컴파일해주면 자동으로 모든것을 알아서 해준다. 여기서, gcc로 컴파일하면 자동으로 LD를 사용하는것처럼 된다.

```
% gcc -o hello hello.o
```

1.4 GAS 사용법 : hellow.S 만들기

앞 강좌를 통해 간단한 GCC에서 추출한 어셈블리어를 컴파일하고, 실행해 보았는데,
이번에는 직접 GAS를 사용한 예제를 만들어 보고, 이것을 컴파일하고 실행해 보기로 한다.

다음은 GAS로 만든 hello.S라는 파일로 화면에 "Hellow World!"를 출력하는 어셈블리로 된 프로그램이다.

```
#
# File : hello.S
# Author : Byungsoo Jung
# Date    : Aug 8, 1999
#
# http:/alpha.secsm.org
#
#
#

.data                   # section declaration

msg:
        .string     "Hello, world!\n"          # our dear string
msgend:
        .equ        len,   msgend - msg        # length of our dear string

.text                   # section declaration

                        # we must export the entry point to the ELF linker or
.global    _start       # loader. They conventionally recognize _start as their
                        # entry point. Use ld -e hello to override the default.

_start:

# write our string to stdout
```

```
movl    $4, %eax      # system call number (sys_write)
movl    $1, %ebx      # first argument: file handle (stdout)
movl    $msg, %ecx    # second argument: pointer to message to write
movl    $len, %edx    # third argument: message length
int     $0x80         # INT 80h

# and exit

movl    $1,%eax       # system call number (sys_exit)
xorl    %ebx,%ebx     # first syscall argument: exit code
int     $0x80         # INT 80h
```

[리스트 1-2] hello.S : 화면에 문자를 출력하는 어셈블리 프로그램

리눅스는 완전한 32비트 운영체제로, 예전의 도스 시절처럼 메모리 모델을 SMALL, LARGE 등과 같이 구별이 없이 4G Byte까지 메모리를 쓰는 완전한 32비트 메모리 모델을 쓰고 있다.

그리고, 예전의 도스에서 문자를 표시하던 도스 함수 Int 21번와 같은 기능을 하는 것이 리눅스의 Int 80번 함수로, 도스 함수와 같은 방식으로 eax에 사용하고자 하는 함수를 써주면 된다. 아직까지 이러한 함수들이 무엇이며, 그 사용법은 어떠하다는 것에 관한 자세한 설명은 없지만, 다음 사이트에 가면 지금 리눅스에서 사용되는 시스템 콜들을 알 수 있다.
http://www.linuxassembly.org/syscall.html

이제 앞의 소스를 잘 살펴보면, 처음 시작을 data segment를 선언함으로써 시작하고 있다. 리눅스는 앞서 말한바와 같이 flat 메모리 모델을 쓰고 있기 때문에, DOS에서와 같이 메모리 모델을 선택할 필요가 없다. 또, 이러한 세그먼트들은 DOS와 비슷해서, data 세그먼트에는 데이터를 text 세그먼트에는 소스 코드를 두면 된다.

프로그램 소스 코드는 text 세그먼트에 두는데, text 세그먼트 안에 다음과 같이 프로그램 시작을 알리는 .global를 세팅한다.

```
                    # we must export the entry point to the ELF linker or
.global  _start     # loader. They conventionally recognize _start as their
                    # entry point. Use ld -e hello to override the default.
```

그 다음에, 미리 만들어 놓은 $msg라는 문자열을 다음과 같이 INT 80 서비스를 이용하여 출력하면 된다.

```
movl    $4, %eax      # system call number (sys_write)
movl    $1, %ebx      # first argument: file handle (stdout)
movl    $msg, %ecx    # second argument: pointer to message to write
movl    $len, %edx    # third argument: message length
int     $0x80         # INT 80h
```

위와 같이 INT80 서비스중에 4번 함수는 sys_write 함수이고, eax에 시스템함수 번호를 적고, ebx에 파일 핸들과 , ecx에 메시지 포인터, edx에 메시지 문자열의 길이를 적고,

INT80함수를 호출하면 된다.

INT80번 함수 중에 1번 함수는 프로그램 종료함수이며, 프로그램 마지막에 다음과 같이 호출하여 프로그램을 종료한다.

```
movl    $1,%eax         # system call number (sys_exit)
xorl    %ebx,%ebx       # first syscall argument: exit code
int     $0x80           # INT 80h
```

이상으로, 프로그램을 작성하였으면, 이것을 컴파일해야 하는데, GAS 어셈블러인 as를 사용하여, OBJ 파일을 만든다.

```
% as -o hello.o hello.S
```

그리고, 로더인 ld로 ELF 포맷인 실행화일을 만든다.

```
% ld -s -o hello hello.o
```

위 두 개의 단계를 한꺼번에 할 수 있는 Makefile이 준비되어 있으니, make를 사용할 수 있으면 다음과 같이 make를 실행하고, hello를 실행하면 된다.

1.5 인라인 어셈블리

다음으로 C 언어에서 인라인 어셈블리하는 방법으로, 만약 다음과 같은 어떤 어셈블리어 한 줄을 C 언어로 된 프로그램에 첨가하고 싶다고 하자.

```
movl %ecx, %ebx
```

위와 같은 어셈블리어는 다음과 같은 asm()라는 키워드로 인라인 어셈블리할 수 있으며 asm 키워드는 다음과 같이 정의되어 있다.

```
__asm__("instructions" : outputs : inputs : modified registers);
```

즉 위에서 쓰고 싶었던 인스트럭션을 처음에 써주고, 인력이 되는 레지스터와 출력이 되는 레지스터, 그리고, 바뀐 레지스터를 써주면 된다. 하지만 위에서 마지막 3부분은 안써도 된다. 즉, 우리가 하고자 하는 위 어셈블리어 루틴은 다음과 같이 간단히 C에서 쓰면 되는 것이다.

```
__asm__("movl %eax, %ebx");
```

다음은 위 어셈블리어 루틴이 실제 인라인 어셈블리어로 프로그램된 조그마한 예제이다.

```
/*
 * File : add.c
 * Author : Byungsoo Jung
 * Date   : Aug 8, 1999
 *
 * http:/alpha.secsm.org
 */

#include <stdio.h>

int add(int i, int j)
```

```
{
    int k=0;
  __asm__ __volatile__("
    pushl %%eax\n
    movl %1, %%eax\n
    addl %2, %%eax\n
    movl %%eax, %0\n
    popl %%eax\n"
        : "=g" (k)
        : "g" (i), "g" (j)
  );
  return k;
}

int main()
{

int i, j;

i = 2;
j = 3;

printf("i = %d\n", i);
printf("j = %d\n", j);

printf("%d + %d =  %d\n", i,j,add(i,j));

return 0;
}
```

[리스트 1-3] add.c : 인라인 어셈블리어를 사용한 예제 프로그램

위 프로그램은 어느 두 숫자를 더한 값을 출력하는 간단한 프로그램으로, 덧셈을 하는 부분을 인라인 어셈블리 언어로 만든 함수를 이용하고 있다.

인라인 어셈블리는 전체 프로그램에서 빠른 속도를 요구하는 부분이나(게임 프로그램의 그래픽에 관한 부분이나, FFT연산등에 사용된다.) 시스템영역 깊숙히 제어를 하고싶을 때, 굳이 어셈블리어만 따로 만들지 않고, 전체 C 프로그램에서 필요한 부분만 어셈블리어로 만들 때 유용하게 쓰이고 있다.

위 예제에서의 다음과 같은 부분이 인라인 어셈블리어로 된 부분이다.

```
__asm__ __volatile__("
    pushl %%eax\n
    movl %1, %%eax\n
    addl %2, %%eax\n
```

```
    movl %%eax, %0\n
    popl %%eax\n"
        : "=g" (k)
        : "g" (i), "g" (j)
);
```

위 인라인 어셈블리어는 앞의 인라인 어셈블리어의 형식을 잘 따르고 있는데 그 내용을 차근차근 살펴보면 다음과 같다.

__volatile__ 의 의미는 뒤에 만든 어셈블리어를 GCC에서 최적화하는 과정에서 그 순서나 내용을 바꾸지 말라는 의미이고, asm()안의 인자들은 ':'의로 구분되어 있다. 첫 부분은 인스트럭션을 적는 부분이고, 두 번째 부분은 출력 인자들을 적고, 세 번째는 입력 인자들을 적고, 네 번째는 변화되는 레지스트러를 적는 부분이다.

asm()안의 첫 번째 부분에서는 우선 eax 레지스터를 스택에 넣어서 그 값을 보존하고, 입력 인자(1번 레지스터)를 eax에 옮겨놓는다. 그 다음에 두 번째 입력 인자(2번 레지스터)와 addl 인스트럭션으로 그 값을 더하여 다시 eax에 보관한다. 이것을 출력 인자(0번 레지스터)에 옮겨 놓으면, 입력 인자들의 합을 출력 인자에 저장할 수 있는 것이다.

두 번째 부분에서는 출력 인자를 지정하는 곳으로, '='는 출력 인자를 나타내는 기호이고, 'g'는 컴파일러가 이러한 인자를 어디서 로딩할지를 결정하는 것으로 컴파일러가 임의로 결정하게 하는 것이다. 'r'로 하면 레지스터에서 로딩하는 것이고, 'a'로 하면 eax 레지스터에서 로딩하도록 하는 것이다.
세 번째 부분은 입력 인자를 지정하는 곳이다. 앞의 출력 인자와 같은 방법으로 지정하며, 이때 출력 인자가 1개일 경우에는 어셈블리어에서 0번 레지스터는 출력인자의 값이 들어있고, 나머지 1번 레지스터부터 차례대로, 입력 인자의 레지스터가 되는 것이다.
이상과 같이 프로그램을 작성했으면, 다음과 같이 일반적으로 C 언어 컴파일하듯이 컴파일하고, 실행하면 된다.

```
% gcc -O -o add add.c
% add
i = 2
j = 3
2 + 3 =  5
```

맺음말

이상으로 간단히 GAS 사용법을 알아보았다. 현재 GAS를 배우는 사람들은 더할나위 없이 좋은 환경에서 행복한 시절에서 프로그래밍을 하고 있는 셈이다. 예전에는 GNU에서 제공하는 GAS 매뉴얼 하나에 의지해서, 모든 것을 했으나, 이제는 linuxassembly.org를 주축으로 다양한 초보자를 위한 문서화 프로젝트와 어셈블리어로 된 각종 유틸리티가 만들어지고 있는 실정이다.
늘 어셈블리어가 좋은 필자에게 GAS처럼 좋은 친구가 없는 것 같다. 잘만 익히면, i386뿐만 아니라, ARM, MIPS, PPC등 다양한 CPU를 위한 시스템 프로그래밍을 할 수도 있고, 누군가가 컴파일러를 만들어주기 전에 새로운 CPU에도 GAS를 적용하여 프로그래밍할 수도 있으니 말이다.

2. 간단한 데이터 연산예제

컴퓨터가 처음 생기게 되는 원인이 바로 계산 때문이다. 즉, 숫자 계산에 탁월한 기계가 바로 컴퓨터인 것이다. 이러한 간단한 숫자를 컴퓨터가 어떻게 취급하는 것인가를 배우는 것이 이번 강좌의 핵심이다.
이번 강좌를 통해 우리는 컴퓨터에서 숫자를 어떻게 취급하고 있으며, 간단한 사칙 연산 프로그램을 통하여, 그 사용법을 익히고, 결과 확인을 위해 GDB를 사용해 보기로 한다.

2.1 컴퓨터에서의 숫자

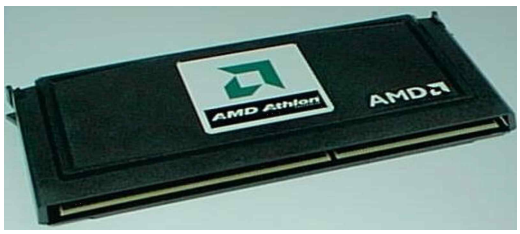
컴퓨터에서 숫자는 크게 10진수와 2진수, 그리고 16진수를 쓰고 있다.
10진수는 우리가 평소 쓰는 숫자로 밑이 10인 수로 0부터 9까지의 숫자들로 이루어진 체계이다. 우리가 쓰고 있는 10진수는 인간의 손가락이 10개이기 때문이라는 설이 유력하지만, 예외도 있어서, 어느 나라에서는 예전에는 16진수를 썼다고 한다.
컴퓨터는 기계여서, 0과 1이라는 숫자밖에 인식하지 못하여, 2진수를 쓰고 있다. 즉, 우리가 10진수로 23이라는 숫자는

23 = 2 x 10^2 + 3 x 1 = 1 x 2^4 + 0 x 2^3 + 1 x 2^2 + 1 x 2 + 1

이 되어서, 10진수 23은 2진수로 10111이 된다. 또, 우리는 보통 0과 1의 나열인 2진수로 된 컴퓨터의 숫자를 보기 위해 10진수와 비슷한 16진수로 바꾸어서 2진수를 읽고 있으며, 앞의 1 0111은 4개씩 끊어서 읽으면 1과 0111이 되며, 16진수로는 17이 되어 인간이 읽기 편하게 하고 있다.
일반적으로 컴퓨터이 모든 숫자들은 16진수로 표현하고 있으며, 특히 메모리나 데이터들을 취급할 때, 16진수를 자주 쓰게 된다. 하지만 비트 연산을 취급할때는 종종 2진수를 쓰고 있으니 참조하기 바란다.
또 참고로 앞의 4비트를 니블(nibble)이라고 하며, 8비트를 바이트(byte)라고 한다. 보통 컴퓨터의 모든 데이터는 바이트 단위로 표현하며, 메모리의 가장 기본적인 단위가 된다. 바이트보다 큰 숫자는 워드(2바이트, 16비트), 더블 워드(4바이트, 32비트), 쿼드 워드(8바이트, 64비트), 파라그래프(16바이트, 128비트)가 있다.

2.2 CPU의 구조

우선 우리는 컴퓨터에서 숫자를 다루는 것을 알아보기 전에 먼저 CPU의 구조에 대해서 먼저 알아 봐야겠다.



[그림 2-1] AMD의 1GHz Athelon

우리가 공부하는 CPU인 인텔의 80x86은 16비트 컴퓨터이고, 80x386은 32비트 컴퓨터이다. 인텔 CPUCPU는 처음 4비트에서 8비트 16비트, 32비트로 순차적으로 만들었으며, 현재는 IA64로 64비트까지 만들고 있다.
하지만 요즘 Intel의 가장 큰 경쟁자인 AMD의 Athlon CPU가 이틀 일찍 1GHz의

테이프를 끊자 Intel의 위상이 한풀 꺾인 모습이다.

앞서 말한 80x86은 16비트이고, 최대 지정할 수 있는 메모리는 20비트로 1MB밖에 되지 않는다. 이러한 모드를 리얼 모드라고 하고, 80x386이후에는 32비트 어드레스 모드를 가지므로 최대 4GB까지 메모리를 가질 수 있는 것이다.

우리는 이러한 CPU에 있는 레지스터에다가 숫자를 집어넣고 연산을 해야하는데, Intel의 경우에는 다음과 같이 4개의 범용 레지스터가 준비되어 있다.

16비트	32비트
AX : 누산 레지스터	EAX
BX : 베이스 주소 레지스터	EBX
CX : 카운터 레지스터	ECX
DX : 데이터 레지스터	EDX

위의 레지스터를 다룰 때 16비트일 때의 인스트럭션과 32비트일때의 인스트럭션이 달라지게 되며, 보통 덧셈의 경우, iaddb는 AX를 다시 나눈 AH(상위바이트), AL(하위 바이트)를 다룰 때 쓰이고, iadd는 AX를 다룰 때 쓰이고, iaddl은 EAX를 다룰 때 쓰인다.

2.3 간단한 사칙연산 프로그램

이제 덧셈, 뺄셈, 곱셈, 나눗셈 등 간단한 사칙 연산 프로그램을 어셈블리어로 작성해 보고, 이를 GDB로 확인해 보자.

다음은 두 개의 숫자를 더해서 그 결과를 보는 간단한 덧셈 프로그램이다.

```
#
# File : add.S
# Author : Byungsoo Jung
# Date    : May 8, 2000
#
# http:/alpha.secsm.org
#
#
#
.data                   # section declaration
a :
        .byte 0x32
b :`
        .byte 0x31
c :
        .byte 0

.text                   # section declaration
```

```
        .align 4                # set 4 double work alignment
                                # we must export the entry point to the ELF linker or
.global _start      # loader. They conventionally recognize _start as their
                                # entry point. Use ld -e foo to override the default.

_start:

# move data

_funcall:
        pushl   %ebp
        movl    %esp, %ebp

_add:
        movb    a, %al
        addb    b, %al
        movb    $0, %ah
        movb    %al, c

_exit:
        movl    $1,%eax             # system call number (sys_exit)
        xorl    %ebx,%ebx           # first syscall argument: exit code
        int     $0x80               # call kernel
```

[리스트 2-1] 간단한 덧셈 프로그램 (add.S)

위 [리스트 2-1]은 간단한 리눅스 덧셈 프로그램이다.

2.3.1 컴파일은 다음과 같이 한다.

```
% as --gstabs -o add.o add.S
```

위에서 --gstabs를 두는 이유는 나중에 GDB에서 디버그하기 위해서이다. 우리가 GCC에서 -g를 두는 것과 같은 것이다.

2.3.2 실행 파일은 다음과 같이 LD를 이용하여 만든다.

```
% ld -o add add.o
```

위에서 add.S는 일반적인 glibc라는 라이브러리를 이용하지 않고, 모든 시스템 함수를 어셈블리에서 작성하여 쓰고 있으므로, 자체 오브젝트 파일만으로 완전한 실행 파일을 만들 수 있다.

이제 실행 파일을 만들었으므로, gdb를 사용하여 디버그를 하면서, 프로그램을 분석해 보기로 하자.

gdb를 실행한다.

```
% gdb add
GNU gdb 4.18
```

```
Copyright 1998 Free Software Foundation, Inc.
GDB is free software, covered by the GNU General Public License, and you are
welcome to change it and/or distribute copies of it under certain conditions.
Type "show copying" to see the conditions.
There is absolutely no warranty for GDB.  Type "show warranty" for details.
This GDB was configured as "i386-redhat-linux"...
(gdb)
```

처음 gdb를 실행하면 위와 같이 시스템에 설치되어 있는 gdb 버전과 i386-redhat-linux와 같이 현재 설정되어 있는 gdb의 환경을 보여준다. 여기서는 redhat 6.1에 설치되어 있는 ELF 형식의 gdb를 이용하고 있다.

소스 코드를 확인한다.

```
(gdb) list
1       #
2       # File : add.S
3       # Author : Byungsoo Jung
4       # Date    : May 8, 2000
5       #
6       # http:/alpha.secsm.org
7       #
8       #
9       #
10      .data                   # section declaration
(gdb)
```

위에서 list라는 명령어을 입력하면 현재 우리가 실행하는 프로그램의 소스 코드를 그대로 보여준다.

breakpoint를 설정한다.

```
(gdb) break _start
Breakpoint 1 at 0x8048077: file add.S, line 30.
(gdb)
```

프로그램의 소스를 보면 프로그램의 시작은 _start에서 시작되고 있으므로, 우리는 _start에 breakpoint를 두고, 여기서부터 차근차근 프로그램을 단계적으로 실행하기 위함이다. 참고로 보통 C언어에서 main이라는 함수는 어셈블리어에서 _main과 같이 앞에 '_'가 붙여서 있게 된다.

프로그램을 실행시킨다.

```
(gdb) run
Starting program: /home/bsjung/asm/asm-ex3/add

Breakpoint 1, _add () at add.S:34
34                      movb    a, %al
Current language:  auto; currently asm
(gdb)
```

위와 같이 run이라는 명령어를 사용하여, 디버거 안에서 프로그램을 실행시킨다.

초기 레지스터의 값들을 조사한다.

```
(gdb) info reg
eax            0x0        0
ecx            0x0        0
edx            0x0        0
ebx            0x0        0
esp            0xbffff89c        0xbffff89c
ebp            0xbffff89c        0xbffff89c
esi            0x0        0
edi            0x0        0
eip            0x8048077        0x8048077
eflags         0x246      582
cs             0x23       35
ss             0x2b       43
ds             0x2b       43
es             0x2b       43
fs             0x0        0
gs             0x0        0
(gdb)
```

위와 같이 info register(보통 줄여서 reg)라는 명령어를 사용하여, 초기 레지스터 값을 조사한다. 어느 특정한 레지스터의 값을 알고 싶으면, 예를 들어 'info reg eax'와 같이 하면 eax의 값을 알 수 있다. 또 다른 방법으로 'print /x $eax'와 같이 해도 같은 결과를 얻을 수 있다.

스텝별로 코드를 실행한다.

```
(gdb) s
35              addb    b, %al
(gdb)
```

위에서는 step이라는 명령어로 한줄한줄 실행시킬 수 있는데, 주의할 점은 바로 앞줄의 코드를 실행시킨다는 점이다. 즉 위의 34번째 줄의 코드를 실행하는 것이지 다음 35번째 줄의 코드를 실행하지 않는다는 것이다.

위의 스텝에서 실행한 결과를 확인해본다.

```
(gdb) info reg eax
eax            0x32       50
(gdb)
```

앞의 스텝에서는 다음 34번째 줄의 코드를 실행하는 것이다.

```
34              movb    a, %al
```

또 프로그램에 a는 .data 섹션에 다음과 같이 정의되어 있으므로,

```
a :
        .byte 0x32
```

결과적으로 %al에는 0x32가 오게 된다. 이것을 eax의 레지스터 내용을 봄으로써 확인할 수 있다.

다음 스텝을 실행한다.

```
35                  addb    b, %al
(gdb) s
36                  movb    $0, %ah
(gdb) info reg eax
eax             0x63    99
(gdb)
```

위 스텝이 이 프로그램의 핵심이라고 할 수 있으며, addb라는 i386 인스트럭션을 사용하여, 기존의 eax에 저장되어 있는 0x32에 새로운 값인 b(0x31)을 더한다.

다음 스텝을 실행한다.

```
36                  movb    $0, %ah
(gdb) s
37                  movb    %al, c
(gdb) info reg eax
eax             0x63    99
```

이제 ax의 상위 바이트에 0을 채우는 작업을 한다. 왜냐하면 지금까지 addb는 한 바이트 연산이므로, 상위 바이트가 FF로 채워지면 값이 엉뚱하게 나올 수 있기 때문이다.

다음 스텝을 실행한다.

```
37                  movb    %al, c
(gdb) s
40                  movl    $1,%eax         # system call number (sys_exit)
(gdb)
```

이제 마지막으로 %al에 저장되어 있는 덧셈한 결과를 우리가 원하는 메모리 영역인 c에 저장하면 된다.

덧셈한 결과를 메모리에서 확인해 본다.

```
(gdb) info addr c
Symbol "c" is at 0x8049096 in a file compiled without debugging.
(gdb) p/x c
$1 = 0x63
```

위와 같이 c라는 메모리 영역(0x8049096)에 0x63이 저장되어 있게 된다.

계속 다음 스텝을 실행하여 프로그램을 종료한다.

```
(gdb) s
41                  xorl    %ebx,%ebx       # first syscall argument: exit code
(gdb) s
42                  int     $0x80           # call kernel
(gdb) s
```

```
Program exited normally.
(gdb) s
The program is not being run.
```

이제 더 이상 프로그램이 돌아가지 않게 된다. 위에서는 0x80 인터럽트를 사용하여 sys_exit() 시스템 커널 함수를 호출하고 있다.

이제 gdb를 종료한다.

```
(gdb) quit
```

이제 앞의 덧셈 프로그램을 조금 고쳐서 곱셈 프로그램을 만들어 보자. (뺄셈이나 나눗셈은 소스 코드로만 제공하고 있으니, 첨부되는 소스 코드를 참고하기 바란다.)

```
#
# File : mul.S
# Author : Byungsoo Jung
# Date    : May 8, 2000
#
# http:/alpha.secsm.org
#
.data                   # section declaration
a :
        .byte 0x1
b :
        .byte 0x2
c :
        .byte 0

.text                   # section declaration

        .align 4            # set 4 double work alignment
                        # we must export the entry point to the ELF linker or
.global _start     # loader. They conventionally recognize _start as their
                        # entry point. Use ld -e foo to override the default.

_start:

# move data

_funcall:
        pushl     %ebp
        movl      %esp, %ebp

_mul:
        movb      a, %al
        movb      b, %bl
        imul      %bx, %ax
```

```
        movb    %al, c

_exit:
        movl    $1,%eax             # system call number (sys_exit)
        xorl    %ebx,%ebx           # first syscall argument: exit code
        int     $0x80               # call kernel
```

[리스트 2-2] 간단한 곱셈 프로그램 (mul.S)

위 곱셈 프로그램의 핵심은 다음과 같은 루틴으로

```
imul    %bx, %ax
```

ax레지스터의 내용과 bx 레지스터의 내용을 곱하여, 다시 ax레지스터에 넣는 것이다. 이때 주의해야할 것은 처음 ax레지스터와 bx 레지스터에는 1바이트의 숫자만 있어야한다는 점이다. 왜냐하면, 1바이트 숫자와 1바이트 숫자를 곱하면 2바이트 숫자가 되기 때문이다.

그래서 앞의 덧셈의 경우에는 iaddb라는 바이트 연산 인스트럭션을 썼지만, 이번에는 imul이라는 워드 연산 인스트럭션을 쓰고 있는 것이다.

다음은 앞의 덧셈 프로그램에서와 같이 gdb로 디버깅한 내용 중 중요한 부분을 발췌한 것이다.

```
(gdb)   b _start
Breakpoint 1 at 0x8048077: file mul.S, line 30.
(gdb) run
Starting program: /user/bsjung/asm/asm-ex3/mul

Breakpoint 1, _mul () at mul.S:34
34                  movb    a, %al
Current language:   auto; currently asm
(gdb) s
35                  movb    b, %bl
(gdb) s
36                  imul    %bx, %ax
(gdb) info reg eax ebx
eax             0x1         1
ebx             0x2         2
(gdb) s
37                  movb    %al, c
(gdb) info reg eax ebx
eax             0x2         2
ebx             0x2         2
(gdb) s
40                  movl    $1,%eax             # system call number (sys_exit)
(gdb) info addr c
Symbol "c" is at 0x804909a in a file compiled without debugging.
(gdb) p/x c
$1 = 0x2
```

```
(gdb) quit
```

위에서 처음에 eax와 ebx에 0x1과 0x2가 있었다가, imul 인스트럭션을 실행하니까 0x1과 0x2를 곱한 값이 eax에 0x2로 저장하게 되었다. 또 그 값을 메모리 c(0x804909a)에 저장하고 있다.

맺음말

이상으로 리눅스에서 숫자를 가지고 사칙 연산을 하는 법을 알아보았다. 앞 강좌에서는 문자 출력을 위해서 리눅스 커널 함수를 호출하기 때문에 다소 소스 코드가 복잡했지만, 일반적으로 어셈블리어에서는 숫자를 다루는 부분은 매우 간단한 논리 구조를 가지고 있다.

실제적으로 매우 간단한 임베디드 시스템의 경우 커널이 없이 이러한 간단한 숫자 조작만으로도 훌륭한 프로그램을 만들 수가 있는 것이다. 즉, 우리가 마지막에 특정한 메모리영역에 데이터를 써 놓은 것을 조금 바꿔서, 예를 들어 시리얼 포트의 메모리 영역에 데이터를 써 놓으면 외부의 세계와 통신할 수 있는 시스템이 되는 것이다.

3. 간단한 프로시져와 마크로예제

여러분이 처음 컴퓨터란 물건이 세상에 나왔을 때를 상상해 보라. 예를 들어, 폰 노이만이 처음으로 최초의 저장식 컴퓨터(예전에 ENIAC같은 경우는 지금의 샤프의 전자계산기와 같이 계산만을 위해 특화된 병렬화된 컴퓨터였다.)를 조용한 프린스턴 고등과학원에서 만들 때를 생각해 보자. (아인슈타인같은 사람은 이때 같은 프린스턴 고등과학원에서 왜 쓸데없이 컴퓨터란 것을 만들까? 하고 생각했을 것이다. ^.^) 그때 유명한 컴파일러의 개척자인 울람등과 같은 어셈블리어 프로그래머들은 천공 펀치와 같은 기계적인 동작에 의해서 그 당시 기억장치였던 마그네틱 코어에 1비트씩 입력하고 있었다.

이와 같이 천공펀치 입력과 같이 단순한 작업을 계속 하다보면, 같은 작업을 계속하게 되는데, 이것들을 미리 작성해 두고 그것을 불러 쓰고 싶어질 것이다. 초기 어셈블리어 프로그래머들도 이와 같은 생각에서 기계어에 가까운 어셈블리어에서 프로시져와 마크로를 도입하게 되었다. 그리고, 울람과 같은 사람들은 더 상위 언어인 포트란이라는 컴파일러를 만들게 된 것이다.

우리는 보다 편하게 어셈블리어를 하기 위해 필요한 프로시져 사용법과 마크로를 이용하는 방법에 대해서 알아보고자 한다. 이러한 것들을 잘만 이용하면, 고급 언어에서 하던 거의 모든 프로그램을 프러시져를 사용하여 코드를 적게 쓰고도 만들 수 있다. 그럼 어셈블리어를 이용한 DB하나 만들어보면 어떨까 생각해 본다. (^.^)

3.1 어셈블리에서 프로시져 사용법

어셈블리어를 작성하다 보면 똑같은 코드를 작성하는 일이 많은데, 예를 들어 문자를 출력하는 루틴의 경우, 이 루틴을 계속해서 작성하는 것은 시간낭비이다. 이러한 시간 낭비를 피하기 위해서는 프로그램에서 공통으로 쓰는 부분을 프로시져로 작성하여, 이 프로시져를 호출하면 된다. 이렇게 프로시져를 자신의 어셈블리어 프로그램에 자주 쓰는 것이 프로그램을 구조화시키고 나중에 알아보기 쉽게 하는 방법이기도 하다.

그러면 이러한 소프트웨어 공학적인 방법(소프트웨어의 재사용화)을 우리가 처음으로

리눅스에서 어셈블리어를 배우면서 hello.S라는 프로그램을 예제로 사용하여 적용해 보자.

```
#
# File : hello.S
# Author : Byungsoo Jung
# Date    : Aug 8, 1999
#
# http:/alpha.secsm.org
#

.data                   # section declaration

msg:
        .string     "Hello, world!\n"          # our dear string
msgend:
        .equ        len,   msgend - msg        # length of our dear string

.text                   # section declaration

                        # we must export the entry point to the ELF linker or
.global    _start       # loader. They conventionally recognize _start as their
                        # entry point. Use ld -e hello to override the default.

_start:

# write our string to stdout

movl    $4, %eax        # system call number (sys_write)
movl    $1, %ebx        # first argument: file handle (stdout)
movl    $msg, %ecx      # second argument: pointer to message to write
movl    $len, %edx      # third argument: message length
int     $0x80           # INT 80h

# and exit

movl    $1,%eax         # system call number (sys_exit)
xorl    %ebx,%ebx       # first syscall argument: exit code
int     $0x80           # INT 80h
```

[리스트 3-1] hello.S : 화면에 문자를 출력하는 어셈블리 프로그램

위 프로그램은 본 강좌에서 가장 처음 시작할 때 사용한 예제로, 화면에 문자를 출력하는 어셈블리 프로그램이다. 이 프로그램을 이제 화면 출력부분과 프로그램 종료부분을 프로시져를 써서 마크로와 서브루틴을 써서 간단히 할 수 있다.
어셈블리 프로그램은 크게 .text 세그먼트와 .data 세그먼트, .bss 세그먼트로 되어 있다. 간단한 프로그램일 때는 .text 세그먼트에 어셈블리 코드를 두고, .data 세그먼트에 초기화된

데이터를 두고, .bss 세그먼트에 초기화되지 않은 스택 데이터를 두면 된다.

그러면 프로그램의 처음에 .data 라는 의사명령어로 data 세그먼트임을 알리고, msg 라는 주소에 "Hello, world!\n" 라는 문자열을 저장해 두고 있다.

```
.data                   # section declaration

msg:
        .string     "Hello, world!\n"       # our dear string
msgend:
        .equ        len,  msgend - msg      # length of our dear string
```

위의 msgend라는 주소를 다시 지정하여, 문자열의 길이를 msgend와 msg의 길이의 차이로 계산하고 있다.

앞의 sys_write 함수의 경우는 다음과 같다.

sys_write
Syntax: ssize_t sys_write(unsigned int fd, const char * buf, size_t count)
Source: fs/read_write.c
Action: write to a file descriptor
Details:

위의 시스템 함수 sys_write 함수를 보면 앞의 어셈블리 언어에서와 같이, eax에 시스템 함수 번호 4를 집어넣었고, ebx는 파일 핸들에 해당하는 fd가 ecx에는 문자열의 메시지 포인터 buf가 edx에는 문자열의 길이 count가 각각 할당되어 있다. 즉, 우리는 어셈블리어 프로그래밍을 할 때 시스템 함수 콜의 형식(Syntax)를 보고, eax, ebx, ecx, edx를 사용해서, 인사들을 시스템 함수에 넘겨주고, 인터럽트 80번을 요청하면 되는 것이다. 인자수가 많을 때는 스택을 이용하며, 그 사용법은 다음 강좌에서 다룰 예정이다.

위 프로그램을 컴파일하는 방법은 GAS를 사용하여, 오브젝트 파일을 만들고, 링커 스크립트인 LD를 사용하여, ELF 파일을 만들면 된다.

```
% as --gstabs -o hello.o hello.S
```

위에서 --gstabs를 두는 이유는 나중에 GDB에서 디버그하기 위해서이다. 우리가 GCC에서 -g를 두는것과 같은 이유이다.

```
% ld -o hello hello.o
```

위처럼 직접 ld를 사용하여 실행 파일인 hello를 만들면 된다.

```
% strip hello
```

위 명령어는 hello 실행파일에서 심벌 이름을 벗기는 작업으로 실행 파일의 크기를 줄일수 있다.

[그림 3-2] Art of Assembly 로고

3.2 간단한 마크로 프로그램

이제 앞의 간단한 문자를 출력하는 hello.S라는 어셈블리 프로그램을 MACRO를 사용하여, 구조화된 어셈블리어 프로그래밍을 해보자.

```
#
# File : hello.S
# Author : Byungsoo Jung (bsjung@samsung.com)
# Date    : Dec 8, 2000
#
# http:/alpha.secsm.org
#
.data                   # section declaration

.MACRO print message,len
        movl    $4,%eax         # system call number (sys_write)
        movl    $1,%ebx         # first argument: file handle (stdout)
        movl    $\message,%ecx     # second argument: pointer to message to write
        movl    $\len,%edx      # third argument: message length
        int     $0x80           # call kernel
.ENDM

msg:
        .string     "Hello, world!\n"          # our dear string
msgend:
        .equ        len,   msgend - msg        # length of our dear string

.text                   # section declaration

                        # we must export the entry point to the ELF linker or
    .global _start     # loader. They conventionally recognize _start as their
                        # entry point. Use ld -e foo to override the default.

 _start:

    # write our string to stdout

      print msg,len      # use of print macro with a argument of msg

    # and exit
```

```
        movl    $1,%eax             # system call number (sys_exit)
        xorl    %ebx,%ebx           # first syscall argument: exit code
        int     $0x80               # call kernel
```

[리스트 3-2] hellomacro.S : 화면에 문자를 출력하는 어셈블리 프로그램

위의 프로그램은 단순히 프로그램을 순차적으로 실행하는 hello.S를 조금 발전시켜서, 문자열을 출력하는 부분을 다음과 같이 간단한 MACRO를 만들어서 프로그램을 단순화시켰다.

```
.MACRO print message,len
        movl    $4,%eax         # system call number (sys_write)
        movl    $1,%ebx         # first argument: file handle (stdout)
        movl    $\message,%ecx  # second argument: pointer to message to write
        movl    $\len,%edx      # third argument: message length
        int     $0x80           # call kernel
.ENDM
```

위 MACRO는 자주 쓰는 문자열 출력을 위한 전용 MACRO로 문자열 포인터 message와 문자열 길이 len를 인수로 넘겨서, 내부적으로 sys_write 시스템 함수를 호출하고 있다.

이와같이 MACRO를 만들어 놓고, 프로그램 안에서, 다음과 같이 MACRO를 쓰면 된다.

```
print msg,len       # use of print macro with a argument of msg
```

즉, 이렇게 함으로써, 문자열을 출력할 때마다 인터럽트 80번 호출에 각각의 레지스터를 셋팅하지 않고, 단순히 print 라는 MACRO를 이용하면 편리하게 문자열을 출력할 수가 있다.

또, 이러한 MACRO들을 따로 파일로 만들어 놓고, #include라는 의사명령어를 사용하면 더욱더 프로그램이 간단해진다.



[그림 3-3] 어셈블리어 프로그래밍 저널

3.3 간단한 서브루틴 프로그램

그러면 모든 프로그램에서 반복되는 것들을 MACRO를 사용하여 고쳐야할까? 사실 MACRO는 간단하고 건너야할 인수가 작은 프로시져일 때 적당하고, 보통 복잡한 프로시져일 때는 서브루틴을 만들어 함수 콜을 하는 것이 보편적이다.

다음은 처음의 hello.S를 서브루틴을 만들어 함수콜을 사용하여 간단하게 한 경우이다.

```
#
# File : hello.S
```

```
# Author : Byungsoo Jung (bsjung@samsung.com)
# Date    : Dec 8, 2000
#
# http:/alpha.secsm.org
#
#
#
.data                   # section declaration

msg:
        .string     "Hello, world!\n"          # our dear string
msgend:
        .equ         len,   msgend - msg        # length of our dear string

.text                   # section declaration

                        # we must export the entry point to the ELF linker or
    .global _start    # loader. They conventionally recognize _start as their
                        # entry point. Use ld -e foo to override the default.

 _start:

        call _print   # call the subroutine _print
        nop             # dealy
        call _exit    # call the subroutine _exit

     # write our string to stdout
 _print:
        movl    $4,%eax       # system call number (sys_write)
        movl    $1,%ebx       # first argument: file handle (stdout)
        movl    $msg,%ecx     # second argument: pointer to message to write
        movl    $len,%edx     # third argument: message length
        int     $0x80         # call kernel
        ret                     # return to main procedure

     # and exit
_exit:
        movl    $1,%eax         # system call number (sys_exit)
        xorl    %ebx,%ebx       # first syscall argument: exit code
        int     $0x80           # call kernel
        ret                     # return to main procedure
```

[리스트 3-3] hellosub.S : 화면에 문자를 출력하는 어셈블리 프로그램

위 프로그램을 보면 앞의 복잡했던 문자 출력 루틴과 함수 종료 루틴을 다음과 같이

서브루틴으로 만들었다는 점이다.

```
_print:
        movl    $4,%eax       # system call number (sys_write)
        movl    $1,%ebx       # first argument: file handle (stdout)
        movl    $msg,%ecx     # second argument: pointer to message to write
        movl    $len,%edx     # third argument: message length
        int     $0x80         # call kernel
        ret                   # return to main procedure

      # and exit
_exit:
        movl    $1,%eax       # system call number (sys_exit)
        xorl    %ebx,%ebx     # first syscall argument: exit code
        int     $0x80         # call kernel
        ret                   # return to main procedure
```

즉 위와 같이 각 기능을 하는 서브루틴을 만들고 (항상 마지막에는 ret으로 main으로 돌아가게 한다.) main에서는 다음과 같이 위 서브루틴을 호출만 하면 된다.

```
call _print   # call the subroutine _print
call _exit    # call the subroutine _exit
```

위와 같이 하면, main 루틴이 매우 간결하게 되는 장점이 있다. 즉, 어셈블리어 대부분을 간단한 몇가지의 서브루틴으로 만들 수 있는 것이다.

맺음말

리눅스 어셈블리어를 이용하여, 체계적인 마크로 사용법을 알아보았다. 처음 어셈블리어를 배웠던 선배들은 기계어로 하나하나 펀치하던 것을 이제 어셈블리어를 만들어 간편하게 기계어에서 벗어나더니, 다시 어셈블리어도 반복되는 것을 이와 같이 마크로를 사용하여 편리하게 사용할수 있게 되었다. 요즘에는 이러한 마크로뿐만 아니라, 어셈블리어에서도 if 문이나 while 문 등을 지원한다니, 점차 어셈블리어도 상위 언어를 닮아가고 있다는 느낌을 받는다. 기본인 기계어로 돌아가고 싶은 필자는 아마 구시대의 사람인가보다.

4. 간단한 OS 부트코드 만들기

이번에는 리눅스에서 어셈블리어를 이용해 간단한 OS 부트코드를 만들어 보기로 하자. 간단한 OS는 리눅스처럼 복잡한 OS말고, 처음 DOS1.0이 나왔을 때처럼 간단한 OS를 말한다. 원래 리눅스는 MINIX라는 OS를 기반으로 했고, 학계에서 많은 시간을 투자해서 잘 생각해서 만들었지만, DOS와 같은 간단한 OS는 IBM이라는 고객의 요구에 의해, 시간을 다투는 업계의 요구에 의해 개발되었기 때문에, QDOS라는 이미 업계에 있던 DOS를 이용하고, 또 그 개발자를 고용해서 마이크로소프트는 IBM이라는 큰고객을 만족해야했다. 이러한 마이크로소프트를 세계 최대의 소프트웨어 기업으로 만든 DOS도 시애틀의 조그만 회사를 운영하던 Tim Paterson이라는 엔지니어가 24살 때 만든 것이다. 여러분도 간단한 OS를 자신의 손으로 만들면서 느끼는 희열감을 느꼈으면 한다.

4.1 리눅스에서 NASM 사용법



[그림 4-1] NASM 로고 (주 : nasm.gif)

위 [그림 4-1]은 NASM의 로고로, NASM은 Netwide Assembler의 약자이다. 현재 시스템 해커들이 가장 많이 쓰는 어셈블리어가 바로 GAS와 NASM일 것이다. GAS는 x86뿐만 아니라, Alpha, ARM, PPC등 여러 CPU를 지원하지만, NASM은 x86만 지원한다. 하지만 GAS가 32비트 어셈블리어로 x86의 16비트 초기화루틴을 작성할 수 없기 때문에, 리눅스 커널에서는 AS86이라는 GAS와 다른 또다른 어셈블리어를 사용하여 부팅하는 초기화루틴을 작성하고 있다.
NASM은 GAS와는 달리 x86만 지원하지만, x86에 대해서는 16비트와 32비트 모두 프로그램 할 수 있어서, NASM가지고 16비트 부팅코드와 32비트 어셈블리어를 같이 프로그래밍할 수 있다는 장점이 있다. 그래서, x86용 부팅 코드들은 리눅스를 제외한 대부분은 NASM으로 되어 있는 경우가 많다.
다음은 전에 GAS로 된 hello.S로 된 프로그램을 똑같이 NASM으로 짠 것이다.

```
#
# File : hello.asm
# Author : Byungsoo Jung
# Date    : Aug 8, 1999
#
# bsjung@samsung.com
#
# nasm -f elf hello.asm -o hello.o
# ld -o hello hello.o
#
section .data                                   ;section declaration

msg       db        "Hello World!",13,10,0    ;our dear string
len       equ       $ - msg                    ;length of our dear string

section .text                                   ;section declaration

                          ;we must export the entry point to the ELF linker or
    global _start;loader. They conventionally recognize _start as their
                          ;entry point. Use ld -e foo to override the default.

_start:

;write our string to stdout

          mov       edx,len ;third argument: message length
          mov       ecx,msg ;second argument: pointer to message to write
          mov       ebx,1   ;first argument: file handle (stdout)
          mov       eax,4   ;system call number (sys_write)
```

```
        int     0x80    ;call kernel

;and exit

        mov     ebx,0   ;first syscall argument: exit code
        mov     eax,1   ;system call number (sys_exit)
        int     0x80    ;call kernel
```

[리스트 4-1] hello.asm : 화면에 문자를 출력하는 어셈블리 프로그램

위의 프로그램을 컴파일하려면 다음과 같이 NASM으로 컴파일하면 된다.

```
% nams -f elf hello.asm -o hello.o -l hello.lst
```

위에서 -f는 생성되는 파일 형식을 -o는 결과 파일 그리고, -l은 리스트파일이라고 디버그용으로 기계어와 소스코드를 같이 보여준다.

이제 이 파일을 linker script인 LD로 다음과 같이 실행파일을 만들어 실행하면 된다.

```
% ld -o hello hello.o
% hello
Hello World!
```

위 [리스트 4-1]의 소스코드의 중요부부만 다시 한번 잠시 살펴보면 다음과 같다.

어셈블리 프로그램은 크게 .text 세그먼트와 .data 세그먼트, .bss 세그먼트로 되어 있다. 간단한 프로그램일 때는 .text 세그먼트에 어셈블리 코드를 두고, .data 세그먼트에 초기화된 데이터를 두고, .bss 세그먼트에 초기화되지 않은 스택 데이터를 두면 된다. 따라서, [리스트 1]과 같이 간단한 프로그램은 .data 세그먼트와 .text 세그먼트만 있다.

그 다음에, 미리 만들어 놓은 $msg라는 문자열을 다음과 같이 INT 80 서비스를 이용하여 출력하면 된다. 인터럽트 80번은 리눅스 커널이 시스템 함수를 호출하는 일종의 약속이다.

```
mov     edx,len ;third argument: message length
mov     ecx,msg ;second argument: pointer to message to write
mov     ebx,1   ;first argument: file handle (stdout)
mov     eax,4   ;system call number (sys_write)
int     0x80    ;call kernel
```

위와 같이 INT80 서비스중에 4번 함수는 sys_write 함수이고, eax에 시스템함수 번호를 적고, ebx에 파일 핸들과 , ecx에 메시지 포인터, edx에 메시지 문자열의 길이를 적고, INT80함수를 호출하면 된다.

NASM의 설치(리눅스용)

1. 제공된 nasm의 소스코드인 nasm-0.98.tar.gz을 적당한 디렉토리에 푼다.

```
% tar zxvf nasm-0.98.tar.gz
```

2. configure를 이용하여, 환경설정을 한다.

```
% configure
```

3. make를 이용하여, 소스를 컴파일한다.

```
% make
```

4. root 권한으로 make install를 하여, 시스템에 인스톨한다.

```
% su
```

```
# make install
```

5. 디폴트로 /usr/local/bin에 어셈블러인 nasm과 디스어셈블리어인 nasmw가 있게된다.

4.2 간단한 부트코드



[그림 --2] FreeDOS 프로젝트 로고

이제 간단한 부트코드를 만들어서 테스트해보자. 먼저 알아야할 기본적인 사항은 PC의 마더보드는 임베디드 시스템처럼 우리가 일일이 하드웨어 디바이스를 컨트롤해주지 않아도, BIOS라는 것이 있어서, 기본적인 하드웨어 입출력에 관한 것인 이미 BIOS에 프로그래밍 되어 있어서, 프로그래머는 이러한 BIOS 함수들을 가져다 쓰게 되면 된다.

처음 PC의 전원을 켜게되면, Power On Reset이라는 시스템 하드웨어 초기화 과정을 거치고, 바로 BIOS의 int 19h를 호출하여 부트스트럽 함수를 실행하게된다. 이 BIOS의 부트스트럽함수는 플로피 디스크나 하드디스크의 첫 번째 섹터(0번 실린더, 0번 헤더, 1번 섹터, 여기서 주의해야할 것은 실린더나 헤더는 0부터 시작하지만 섹터는 1부터 시작한다.)인 부트 섹터를 메모리에 0000:7C00번지부터 한 섹터 (한섹터의 크기는 512바이트)를 로딩하게 된다. 이때 부트섹터의 확인은 섹터의 마지막에 AA55h가 있나 BIOS가 확인하게 되므로, 섹터의 맨 끝에 AA55h를 적어놓으면 부트섹터를 만들 수 있게 된다. 다음은 앞의 사실을 알고, 옆에 IBM BIOS 함수를 옆에 두고 참조하면서 다음과 같이 부팅하면 메시지를 화면에 뿌리는 부팅코드를 짤 수 있게 된다.

```
;
; Simple OS
; 2001 (c) bsjung@samsung.com
; Feb 17, 2001
; nasm -f bin boot.asm -o boot.bin -l boot.lst
;

segment.text

%define BASE                7c00h

                    org     BASE

Entry:              jmp     short real_start

bootmsg db "Booting Simple OS",13,10
                    db "2001 (c) bsjung@samsung.com",13,10
                    db "Reboot> ",0

getkey:
```

```
                xor ax,ax                   ; wait for key
                int 016h
                ret
;       ---------------------------------------------------------------
reboot:
                db 0EAh                     ; machine language to jump to FFFF:0000 (reboot)
                dw 0000h
                dw 0FFFFh                   ; remember intel is little endian!

print:                              ; Dump ds:si to screen.
                lodsb                       ; load byte at ds:si into al
                or al,al                    ; test if character is 0 (end)
                jz done
                mov ah,0eh                  ; put character
                mov bx,0007                 ; attribute
                int 0x10                    ; call BIOS
                jmp print
done:
                ret

real_start:
                cli                         ; disable interrupt
                xor     ax, ax
                mov     ss, ax              ; initialize stack
                mov     ds, ax
                mov     bp, 7c00h
                lea     sp, [bp-20h]
                sti                         ; enable interrupt

                mov     si, bootmsg
        call    print
        call    getkey
        call    reboot

        ; The boot sector is supposed to have to have 0xAA55 at the end of
        ; the sector (the word at 510 bytes) to be loaded by the BIOS...
                times 510-($-$$) db 0
                dw 0xAA55
```

[리스트 4-2] boot.asm : 부팅하여 화면에 문자를 출력하는 부팅 프로그램

아래 부분은 현재 BIOS의 부트스트럽 함수에 의해 메모리 번지 0000:7C00로 로딩된 부트섹터를 즉시 real_start번지로 점프하여 부팅을 하게 하는 루틴이다.

```
%define BASE            7c00h
                org     BASE
```

```
Entry:              jmp       short real_start
```

실제 real_start에는 다음과 같이 초기 스텍값을 지정한다.

위에서 스텍값을 초기화하기 전에 cli를 사용하여 인터럽트를 사용하지 못하게 했다가, 초기화가 끝난 후에는 인터럽트를 사용하게 해야한다.

```
cli                         ; disable interrupt
xor     ax, ax
mov       ss, ax            ; initialize stack
mov     ds, ax
mov       bp, 7c00h
lea       sp, [bp-20h]
sti                         ; enable interrupt
```

그 다음에, 다음과 같이 si에 메시지 번지를 담고, BIOS의 int 10번의 0E함수를 사용하여, 콘솔에 메시지를 출력하게 된다.

```
mov     si, bootmsg
call      print
call    getkey
call    reboot
```

위에서 getkey는 BIOS의 int 16번의 0번 함수를 사용했고, reboot은 BIOS의 int 19번인 reboot 함수를 사용하지 않고, 직접 기계어를 써주어서, FFFF:0000 번지로 점프하게 했다. 여기서 인텔 CPU는 little endian이므로 FFFF 0000 이 아닌 순서를 바꾼 0000 FFFF로 써주어야 한다.그러면 위 부트 이미지를 첨부한 bios.exe를 이용하여 도스창에서 다음과 같이 부팅 디스크를 만들면 된다.

boot.exe의 사용법을 본다.

```
c:> boot.exe /?
Simple OS 2001 (c) bsjung@samsung.com
사용법 : boot [boot_file] [root_file]
```

플로피 디스크를 넣고 boot를 다음과 같이 실행시킨다.

```
c:> boot boot.bin
```

플로피 디스크로 다시 부팅한다.

4.3 간단한 OS 부트코드









[그림 4-3] Mac OS X 로고와 관련된 BSD 커널들

위 [그림 4-3]는 올해 가장 큰 화제를 만들 Mac OS X의 로고이다. Mac OS X의 커널은 BSD 4.4를 기반으로한 FreeBSD와 Mach 커널이라고 한다. Mac OS X의 소소 코드를 Darwin이라는 코드명으로 공개하고 있으니 참고바란다. 이러한 커널은 매우 덩치가 크므로 앞서 우리가 짰던 부트섹터만으로는 부족하다. 그래서 이러한 커널을 root 이미지로 해서 디스크에 놓고, 그것을 부트로더가 읽어서 수행하는 것이 OS의 부트코드의 핵심인 것이다.

디스크에서 커널인 root 이미지를 읽으려면, 보통 디스크에 파일시스템을 만들지만, 간단한 OS에 처음부터 파일시스템까지 고려한다면 무척 고달픈 일일 것이다. 그래서, 첫 번째 섹터에는 부트섹터를 두고, 두 번째 섹터 한 섹터에만 root 이미지가 있다고 가정하고, OS 부트로더를 다음과 같이 설계해 보자.

```
;
; Simple OS
;
; 2001 (c) bsjung@samsung.com
;
; Feb 17, 2001
;
; nasm -f bin bootsec.asm -o bootsec.bin -l bootsec.lst
;

segment         .text

%define BASE                7c00h

        bit16
          org     BASE

Entry:    jmp     short real_start
          nop

bootmsg     db "Booting Simple OS",13,10,"2001 (c) bsjung@samsung.com",13,10,0
loadmsg     db "Loading Simple OS",13,10,0
jmpmsg      db "Jump to the root image",13,10,0
bootdrv     db 0        ; This variable will contain the bootdrive id

real_start:
          cli                           ; disable interrupts
            mov [bootdrv],dl            ; save drive number
            mov ax,0x9000               ; put stack at 0x98000
            mov ss,ax
            mov sp,0x8000
            sti                         ; enable interrupts
```

```
                mov  ax,0x0000
                mov  es,ax
                mov  ds,ax

                mov  si,bootmsg                 ; display load message
                call print

                mov  si,loadmsg                 ; display load message
                call print

                call load                       ; load the root image

                mov  si,jmpmsg                  ; display load message
                call print

                mov  ax,0x8000                  ; set segment registers and jump
                mov  es,ax
                mov  ds,ax
                push ax
                mov  ax,0
                push ax
                retf

;-------------------------------------------------------------------
; Load a program from the bootdrive
load:
                ; Reset the diskdrive (Interrupt 13h, 0)
                push ds                         ; Store DS
                mov  ax, 0                      ; Select function (Reset Disk)
                mov  dl, [bootdrv] ; Drive to reset
                int  13h                        ; Reset it!
                pop  ds                         ; Restore DS
                jc   load                       ; Failed -> Try again

load1:
                mov  ax,8000                    ; ES:BX = 8000:0000
                mov  es,ax
                mov  bx, 0

                ; Read disksectors (Interrupt 13h, 2)
                mov  ah, 2                      ; Select function (Read disk sectors)
                mov  al, 1                      ; read 1 sectors
                mov  ch, 0                      ; cylinder=0
                mov  cl, 3                      ; sector=2
```

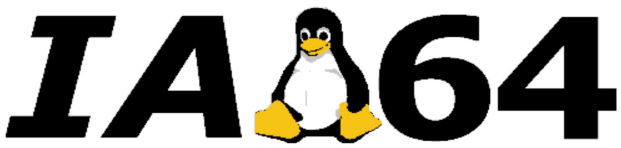
```
                mov dh, 0                   ; head=0
                mov dl, [bootdrv] ; drive=0
                int 13h                     ; ES:BX = data from disk

                jc load1            ; failed -> Try again
                retn

;-------------------------------------------------------------
getkey:
                xor ax,ax                   ; wait for key
                int 016h
                ret
print:                                      ; Dump ds:si to screen.
                lodsb                       ; load byte at ds:si into al
                or al,al                    ; test if character is 0 (end)
                jz done
                mov ah,0eh                  ; put character
                mov bx,0007                 ; attribute
                int 0x10                    ; call BIOS
                jmp print
done:
                ret

        ; The boot sector is supposed to have to have 0xAA55 at the end of
        ; the sector (the word at 510 bytes) to be loaded by the BIOS...
                times 510-($-$$) db 0
                dw 0xAA55
```

[리스트 4-3] bootsec.asm : 부팅하여 화면에 문자를 출력하는 부팅 프로그램



[그림 4-4] IA-64의 로고 (IA-64는 x86과는 전혀 다른 VLIW구조를 가진다.)

위 [리스트 4-3]은 다음의 프로그램 구조로 이루어져 있다. 즉 load 서브루틴에서 BIOS int 13의 2번 함수를 이용하여 한 섹터를 메모리 8000:000 번지에 올리고, retf라는 far return 인스트럭션을 이용해서, stack에 있는 8000과 0을 8000:0000으로 인스트럭션 포인터인 IP를 바꿔서 그 번지로 점프하여 실행하는 것을 나타낸다.

```
call load                               ; load the root image

mov si,jmpmsg                   ; display load message
call print
```

```
mov ax,0x8000                   ; set segment registers and jump
mov es,ax
mov ds,ax
push ax
mov ax,0
push ax
retf
```

즉, 메모리 8000:000에는 다음과 같은 root 이미지가 오게 되어 만들면 된다.

```
;
; Simple OS
;
; 2001 (c) bsjung@samsung.com
;
; Feb 17, 2001
;
; nasm -f bin root.asm -o root.bin -l root.lst
;

root_start:
                mov ax, 8000h               ; Update segment registers
                mov ds, ax
                mov es, ax

                mov     si, rebootmsg
                call    print
                 call    getkey

                 call    reboot

rebootmsg       db "Reboot> ",0

getkey:
                xor ax,ax                   ; wait for key
                int 016h
                ret



;-----------------------------------------------------------------
reboot:
                db 0EAh                      ; machine language to jump to FFFF:0000 (reboot)
                dw 0000h
                dw 0FFFFh                   ; no ret required; we're rebooting!

print:                                      ; Dump ds:si to screen.
```

```
                lodsb                   ; load byte at ds:si into al
                or al,al                ; test if character is 0 (end)
                jz done
                mov ah,0eh              ; put character
                mov bx,0007             ; attribute
                int 0x10                ; call BIOS
                jmp print
done:
                ret
```

[리스트 4-4] root.asm : 부팅하여 화면에 문자를 출력하는 부팅 프로그램

위 [리스트 4-4]에서는 다음과 같이 각 레지스터를 메모리번지에 올바르게 놓으면 되는 것이다. 즉 org 8000h와 같은 것으로 root 이미지가 메모리번지 8000:0000에 오도록 셋팅해주고, 다음 우리가 하고 만들고 싶은 커널 이미지를 구현하면 되는 것이다.

```
mov ax, 8000h           ; Update segment registers
mov ds, ax
mov es, ax
```

그러면 이제 부트 이미지와 루트 이미지를 이용하여 첨부한 bios.exe를 이용하여 도스 창에서 다음과 같이 부팅 디스크를 만들면 된다.

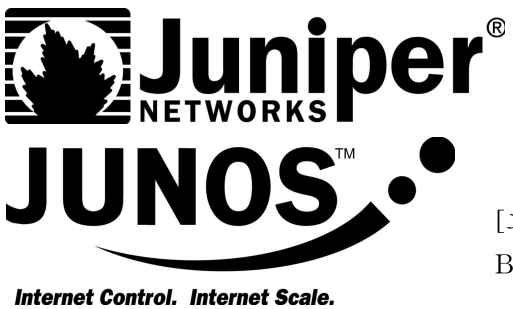
boot.exe의 사용법을 본다.

```
c:> boot.exe /?
Simple OS 2001 (c) bsjung@samsung.com
사용법 : boot [boot_file] [root_file]
```

플로피 디스크를 넣고 boot를 다음과 같이 실행시킨다.

```
c:> boot bootsec.bin root.bin
```

플로피 디스크로 다시 부팅한다.



[그림 4-5]
BSD 커널로 된 Juniper Networks의 JUNOS

맺음말

끝으로 [그림 4-3]의 예쁜 매킨토시의 OS가 BSD인데, 지난 2월 28일 KOEX에서 Mac OS X 세미나에 참석했는데, 발표하는 사람 능력이 대단했다. 한국어와 일본어를 자유자재로 타이핑할 줄 아는 사람이었다. 하지만 영어가 조금 서툴던데, 아마 이스라엘 사람 같다는 느낌이었다. 이 세미나에서 필자는 하나의 데스크탑 OS가 만들어지기까지 커널뿐만 아니라, UI 그리고, 다국어 지원이라는 엄청난 노력이 필요하다는 느낌을 받았다. CISCO의 강력한

라이벌 Juniper Networks가 쓰는 BSD를 앞으로 나올 IA-64에 포팅해 보는 것이 요즘 필자의 머릿속에 있는 생각이다. ^.^

참고문헌

1. ssmLUG GNU-Win32 홈페이지 : http://alpha.secsm.org/gnuwin32
ssmLUG의 GNU Win32 홈페이지로, CrossGCC와 GCC로 윈도우 프로그래밍하는 방법등 GCC에 관한 정보를 제공하는 홈페이지이다.
2. LinuxAssembly.org 홈페이지 : http://linuxassembly.org
현재 GAS HOWTO 문서의 저자인 Konstantin Boldyshev가 운영하는 리눅스에서 어셈블리어 프로그래밍하는 여러 가지 문서와 프로그램들을 모아놓은 곳이다.
3. NASM 홈페이지 : http://www.web-sites.co.uk/nasm/
또다른 i386를 위한 어셈블러로써, 현재 해커들이 GAS다음으로 가장 많이 쓰는 어셈블러이다.
4. Assembly Programming Journal : http://asmjournal.freeservers.com/
각종 어셈블리어 프로그래밍을 하는데 있어서, 필요한 고급 테크닉을 저널 형태로 보여주는 곳이다. MASM 위주로 설명되어있고, 거의 i386만 다루고 있으나, 고급 어셈블리어 기법을 배울수 있다.
5. GAS HOWTO 문서 : http://www.linuxassembly.org/howto/Assembly-HOWTO.html
현재 가장 유용한 GAS에 관한 문서이며, 실질적인 예제도 다루고 있어서 도움이 많이 되는 문서이다. GAS를 배우려면 먼저 이 문서부터 읽어봐야할 것이다.
6. DJGPP 문서 : http://www.delorie.com/djgpp/doc/
MS-DOS에서 실행되는 GCC인 DJGPP의 설명서가 있는 곳으로, GAS에 대한 몇가지 설명서를 구할수 있다.
7. Linux-i386 문서 : http://lightning.voshod.com/asm/
이 문서는 리눅스에서 i386를 위한 어셈블리어를 다룬 문서로써, GAS Howto 문서다음으로 가장 충실하게 설명된 문서이다.
8. Art of Assembly Language : http://webster.cs.ucr.edu/Page_asm/ArtOfAsm.html
이 문서는 실제 어셈블리어를 다룬 책 자체를 웹에 올린 것으로, MASM를 가지고, MS-DOS에서 어셈블리어 프로그래밍을 하는 법을 가르쳐주는 책이다. 이 책 수준은 매우 초급부터 고급까지 망라하고 있으며, 특히 어셈블리어로 하드웨어를 컨트롤하는 부분등 시스템 깊숙이 관여하는 부분까지 다루고 있다.
9. IBM-PC의 내부구조 : 피터 노턴 저, 최홍순, 김석환, 박용규 공역, KALA, 1988
이 책은 피터 노턴의 명저로 IBM-PC의 내부 깊숙이, BIOS 함수와 DOS 인터럽트 함수콜에 대한 자세한 설명을 들을 수 있다.
10. 알기쉬운 PC 어셈블리 : 이세원 저, 정보문화사, 1996
이 책은 MSAM를 중심으로 설명했으며, 기본적인 내용부터 아주 전문적인 내용까지 자세한 어셈블리 테크닉을 설명한 명저이다. 특히 예제로 설명된 간단한 응용 프로그램과 잘 정리된 DEBUG 사용법과 DOS 인터럽트와 BIOS 인터럽트는 프로그래밍할 때 무척 유용하다.
11. Creating Utilities With Assembly Laguate 10 Best for the IBM PC & XT :
Steven Holzner, Brady, 1986
이 책은 MASM을 중심으로 고급 어셈블리 테크닉을 예제로 설명한 책이다. 10가지의 예제들은 하나하나 모두 고급 어셈블리 테크닉이 소개되어 있다. 또, 어셈블리어에서 FAT에 다루는 방법과 곱셈 나눗셈을 구현하는 알고리즘을 설명해두고 있다.
12. Windows 어셈블리 & 시스템 프로그래밍 : 권오철, 이영일 역, 에이스출판사, 1994
이 책은 번역본(저자 미상?)이지만, 프로텍트 모드에서의 어셈블리어에 대한 깊이 있는 설명과 윈도우즈3.1에서의 어셈블리어 사용법을 다루고 있다.
13. PC Assembly Language : Paul A. Carter, 2000
이 책은 인터넷에 있는 NASM으로 된 프로텍트 모드에서의 어셈블리어 프로그래밍에 관한 것으로, 스텍과 실수 연산에 대한 설명이 잘되어있다.
http://www.comsc.ucok.edu/~pcarter/pcasm/book.html